

PROYECTO DE CURSO

Documento de Apoyo 05: (Semana 12)

750021C - DESARROLLO DE SOFTWARE II



FACULTAD DE INGENIERÍA
Escuela de Ingeniería de Sistemas y
Computación

FEBRERO - JUNIO / 2026

Pruebas Automatizadas y Análisis Estático

1. Objetivos

- 1.1. Implementar la ejecución automática de pruebas unitarias dentro del pipeline de CI.
- 1.2. Integrar herramientas de análisis estático (Linting) para garantizar la calidad y legibilidad del código.
- 1.3. Configurar el pipeline para que falle (se detenga) si no se cumplen los estándares mínimos de calidad.

2. Información Clave

Para identificar factores de calidad y seguridad, utilizaremos herramientas que auditan el código sin ejecutarlo (Análisis Estático) y que validan su lógica (Pruebas Unitarias):

- **Pytest:** Framework para ejecutar pruebas de software.
- **Flake8:** Analizador que verifica que el código siga los estándares.
- **Test Report:** Formato de salida que permite a Jenkins visualizar cuántas pruebas pasaron y cuáles fallaron.
- **Exit Codes:** Mecanismo por el cual una herramienta le indica a Jenkins que algo falló (un código diferente a 0 detiene el pipeline).

3. Procedimientos

A. Preparación del Entorno de Pruebas

Es necesario asegurarse de que las dependencias de prueba estén en el archivo de requerimientos `requirements.txt`.

Dado que todos los ejemplos asumen el uso de Python, es necesario incluir `pytest` y `flake8`. En casos como el uso de PHP sería necesario incluir `phpunit/phpunit` y `squizlabs/php_codesniffer` en el archivo `composer.json`.

B. Actualización del Jenkinsfile (Nuevas Etapas)

Se deben añadir los bloques de *Lint* y *Test* antes del paso de *Build* o *Deploy*.

Jenkinsfile

```
...
stage('Static Analysis') {
    steps {
        script {
            sh "docker run --rm ${IMAGE_NAME} flake8 .
--exclude=venv"
        }
    }
}

stage('Unit Tests') {
    steps {
        script {
            sh "docker run --rm ${IMAGE_NAME} pytest tests/"
        }
    }
}
...
```

C. Visualización de Resultados en Jenkins

1. Ir a la configuración del Job.
2. Añadir una **Post-build Action** (o usar el bloque `post` en el Jenkinsfile) para publicar reportes de JUnit si se generan archivos XML de resultados.
3. Esto permitirá ver gráficas de progreso de pruebas en el panel de Jenkins.

4. Resultados Esperados

- Cada Pull Request debe mostrar en Jenkins el resultado del análisis de código y las pruebas.
- Si un estudiante sube un código con errores de sintaxis o lógica que "rompa" las pruebas, el pipeline debe marcarse en rojo.

5. Solución de Problemas

Problema	Posible Causa	Solución Sugerida
El pipeline falla pero el código funciona local	Falta instalar una dependencia en el <code>Dockerfile</code> que se usa en el pipeline.	Revisar que el <code>requirements.txt</code> esté actualizado y subido al repo.
Flake8/PHP_CS da demasiados errores	El código no sigue los estándares de estilo (espacios, nombres de variables).	Ejecutar el linter localmente y corregir el formato antes de hacer el <code>git push</code> .
Jenkins no encuentra la carpeta <code>/tests</code>	La ruta definida en el <code>Jenkinsfile</code> no coincide con la estructura de carpetas del proyecto.	Verificar que la carpeta de pruebas esté en la raíz y se llame exactamente como se definió en el script.